

QUICK SORT

* Quick Sort is an Divide and conquer algorithm.

Steps:

1. Select a pivot element. Pivot means target. Any element can be taken as pivot.
2. Partition operation :- Partition means divide.
 - partition operation divide the array into '2' subarray. [left & right subarray]
 - partition operation places the point element at proper position.
 - That means all element before pivot is smaller & all element after pivot is greater.
3. Recursively, apply quicksort to subarrays [left subarray & Right subarray].

Rules:

1. The value of i is incremented till $a[i] \leq \text{pivot}$.
 2. The value of j is decremented till $a[j] > \text{pivot}$, this process is repeated until $i < j$
 3. If $a[i] > \text{pivot}$ & $a[j] < \text{pivot}$ and also if $i < j$ then
Swap $a[i]$ & $a[j]$
 4. If $i > j$ then swap $a[j]$ and $a[\text{pivot}]$
- ∴ Once the correct location for pivot is found, then partition array into left sub array contains all elements less than

Pivot & right subarray contains all the elements greater than the pivot

Example:

consider an unsorted array as follows

40 20 70 14 60 61 97 80

Sol: Here

Pivot = 40

i = 20

j = 80

Step 1:

40	20	70	14	60	61	97	30
0	1	2	3	4	5	6	7
↑	↑						↑
Pivot	i						j

$i < \text{pivot} \Rightarrow 20 < 40$ [True] i value incremented
 $j > \text{pivot} \Rightarrow 30 > 40$ [false] Not decremented j value

$i = i + 1 \Rightarrow i = 1 + 1 \Rightarrow \boxed{i = 2}$

Step 2:

40	20	70	14	60	61	97	80
0	1	2	3	4	5	6	7
↑		↑					↑
Pivot		i					j

$i < \text{pivot} \Rightarrow 70 < 40$ [false] Not incremented
 $j > \text{pivot} \Rightarrow 80 > 40$ [false] Not decremented

∴ Swap $a[i]$ & $a[j]$

$a[i] = 80$ & $a[j] = 70$.

Step 3:

2.20

0	1	2	3	4	5	6	7
40	20	30	14	60	61	97	70
↑ pivot		↑ i					↑ j

$i < \text{pivot} \Rightarrow 30 < 40$ [True] i value incremented.

$j > \text{pivot} \Rightarrow 70 > 40$ [True] j value decremented

$i = i + 1$ $j = j - 1$

$i = 2 + 1$ $j = 7 - 1$

$i = 3$ $j = 6$

Step 4:

0	1	2	3	4	5	6	7
40	20	30	14	60	61	97	70
↑ pivot			↑ i			↑ j	

$i < \text{pivot} \Rightarrow 14 < 40$ [True] i value incremented

$j > \text{pivot} \Rightarrow 97 > 40$ [True] j value decremented

$i = 3 + 1 = 4$; $j = 6 - 1 = 5$

Step 5:

0	1	2	3	4	5	6	7
40	20	30	14	60	61	97	70
↑ pivot				↑ i	↑ j		

$i < \text{pivot} \Rightarrow 60 < 40$ [false] i not incremented

$j > \text{pivot} \Rightarrow 61 > 40$ [True] j value decremented.

$j = 5 - 1$

$j = 4$

Step 6:

0	1	2	3	4	5	6	7
40	20	30	14	60	61	97	70
↑ pivot				↑ i, j			

$i < \text{pivot} \Rightarrow 60 < 40$ [false] i not incremented.

$j > \text{pivot} \Rightarrow 60 > 40$ [True] j value decremented; $j = 4 - 1$
 $j = 3$

Step 7:

40	20	30	14	60	61	97	70
0	1	2	3	4	5	6	7
↑			↑	↑			
Pivot			j	i			

i & j cross each other

$i < \text{pivot} \rightarrow 60 < 40$ [false] i value not incremented

$j > \text{pivot} \rightarrow 14 > 40$ [false] j value not decremented

$\therefore i > j \Rightarrow 60 > 14$ swap $a[j]$ & $a[\text{pivot}]$

$a[j] = 40$ $a[\text{pivot}] = 14$

Step 8:

14	20	30	40	60	61	97	70
0	1	2	3	4	5	6	7
↑			↑	↑			
j			Pivot	i			

$\therefore$ pivot element reached to correct position.

$\therefore$ Therefore traverse left sub list & right sub list separately.

$\therefore$ Then apply quick sort for left & right sub list again.

left sub list:

14	20	30
↑	↑	↑
Pivot	i	j

$\rightarrow$ Here all the elements are pointed at correct position

$\rightarrow$ Therefore No traversal required.

Right sub list:

60	61	97	70
0	1	2	3

Again apply quick sort pivot = 60

i = 61

j = 70

Step 1:

60	61	97	70
4	5	6	7
↑	↑		↑
Pivot	i		j

2.21

$i < \text{pivot} \rightarrow 61 < 60$ [false] i is not incremented

$j > \text{pivot} \rightarrow 70 > 60$ [True] j is decremented ~

$$j = j - 1$$

$$= 7 - 1$$

$$\boxed{j = 6}$$

Step 2:

60	61	97	70
↑	↑	↑	
Pivot	i	j	

$i < \text{pivot} \rightarrow 61 < 60$ [false] i not incremented

$j > \text{pivot} \rightarrow 97 > 60$ [True] j is decremented

$$j = j - 1$$

$$= 6 - 1$$

$$\boxed{j = 5}$$

Step 3:

60	61	97	70
↑	↑		
Pivot	i	j	

$i < \text{pivot} \rightarrow 61 < 60$ [false] i not incremented

$j > \text{pivot} \rightarrow 61 > 60$ [True] j is decremented

$$j = j - 1 \Rightarrow 5 - 1 = 4$$

$$\boxed{j = 4}$$

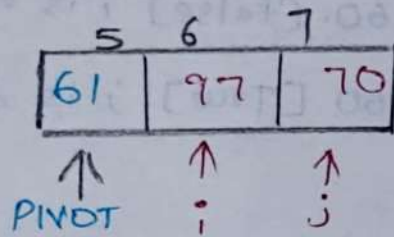
$\therefore$ Pivot & j element points to same element

$\therefore$ Pivot placed at right position.

∴ Traverse 61, 97, 70 again apply quick sort.

61, 97, 70 Travelling

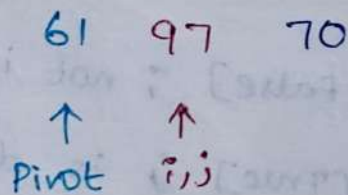
pivot = 61 i = 97 j = 70



i ≠ pivot → 97 < 61 [false] i is not incremented

j > pivot → 70 > 61 [true] j is decremented

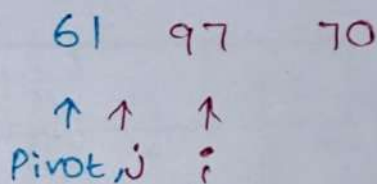
$$j = j - 1 = 7 - 1 = 6$$



i < pivot → 97 < 61 [false] 'i' not incremented

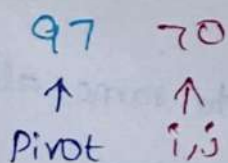
j > pivot → 97 > 61 [true] j is decremented

$$∴ j = j - 1 = 6 - 1 = 5$$



∴ pivot & j points to same position & pivot placed at right position.

Traverse 97, 70



i < pivot → 70 < 97

j > pivot → 97 > 70

∴ i > j, 70 > 70 false swap a[j] & a[pivot]

Swap $a[j]$ & $a[\text{pivot}]$

2.22

$a[i] \leftarrow 97$ $a[\text{pivot}] \leftarrow 70$

70	97
----	----

↑
i

↑
j & pivot

Combine the left & right sublist:

∴ The sorted list of Quick sort is

14	20	30	40	60	61	70	97
----	----	----	----	----	----	----	----

Algorithm for Quick Sort:-

Algorithm qsort ($A[]$, left, right)

{

 if (left < right) then

 {

 pivot ← left

 i ← left + 1

 j ← right

 while (i < j)

 {

 while ($A[\text{pivot}] \geq A[i]$)

 i ← i + 1

 while ($A[\text{pivot}] < A[j]$)

 j ← j - 1

 if (i < j)

 {

 temp ← $A[i]$ // swap

$A[i] \leftarrow A[j]$ // $A[i]$ & $A[j]$

$A[j] \leftarrow \text{temp}$

 }

temp $\leftarrow$ A[PIVOT] // swap A[PIVOT] & A[j]

A[PIVOT] $\leftarrow$ A[j]

A[j] $\leftarrow$ temp

qSort(A, left, j-1)

qSort(A, j+1, right)

}

↓

Analysis of quicksort:-

Worst case analysis = $O(n^2)$

Best-case analysis = $O(n \log n)$

Average-case analysis = $O(n \log n)$

The recurrence relation for quicksort is

$$T(n) = \begin{cases} 2T(n/2) + n & \text{// Best case} \\ T(n-1) + n & \text{// worst case} \end{cases}$$

FOR BEST-CASE & AVERAGE CASE ANALYSIS:

$$T(n) = 2T(n/2) + n$$

By using master method.

Case 2 $a=b \Rightarrow 2=2$ (True)

→ Case '2' satisfies & apply time complexity of quicksort for that case is

$$T(n) = O(n^d \log n)$$

$$\boxed{T(n) = O(n \log n)}$$

FOR WORST CASE ANALYSIS:

$$T(n) = T(n-1) + n$$

Apply backward substitution.

$$T(n) = n \frac{(n+1)}{2} = \frac{n^2 + n}{2}$$

$$T(n) = \Theta(n^2)$$

$\therefore$ It is worst case analysis.